ESPECIFICACIÓN TÉCNICA DE ARQUITECTURA

ATLAS Platform v3.0
Arquitectura Frontend (SPA + Nginx)

Especificación Exhaustiva de Topología de Red, Árbol de Componentes, Gestión de Estados Globales, Cliente HTTP e Interceptores, Seguridad JWT y Directivas de Producción Nginx

COMPONENTE DESACOPLADO **Frontend SPA (atlas-frontend)**	REPOSITORIO GITHUB OBJETIVO **https://github.com/eaguilarh/atlas-frontend.git**
STACK TECNOLÓGICO **HTML5 + JavaScript ES2022 + CSS3 Nativo + SheetJS + Nginx 1.25 Alpine**	FECHA DE ENTREGA Y VERSIÓN **Noviembre 2026 · Versión 3.0.0**

Alcance de la Entrega Técnica: Documento de arquitectura oficial correspondiente a la segregación en repositorios independientes de ATLAS Platform para la operación SAP de Coca-Cola FEMSA (54 sistemas maestros y 1,684 RFCs).

1. Resumen Ejecutivo y Topología de Red

ATLAS Platform (Frontend) es una Single Page Application (SPA) de alta densidad y rendimiento, diseñada para la gestión, supervisión técnica y gobernanza de los 54 sistemas maestros de Coca-Cola FEMSA (KOF) y el control de ventanas de mantenimiento de 1,684 RFCs operacionales. Opera completamente desacoplada de la capa de persistencia mediante un contenedor `nginx:alpine` que provee terminación estática y actúa como proxy inverso hacia el backend REST.



Figura 1.1: Topología de Red Desacoplada del Frontend y Flujo Reverse Proxy hacia Backend API

2. Árbol de Componentes y Módulos de Interfaz (SPA)

La aplicación implementa un diseño modular orientado a componentes montados sobre un shell reactivo con anclaje de viewport

(`width: calc(100vw - 248px)`) y contenedores de desplazamiento horizontal (`.table-scroll`) que eliminan desbordamientos en pantalla.

Componente / Módulo	Responsabilidad Funcional	Roles Habilitados	Estado y Persistencia
App Shell & Sidebar	Navegación lateral responsiva, perfil activo, indicador de estado de conexión y conmutador de modos TQS.	Todos los roles	`window.state.currentPage`
Dashboard Principal	6 KPI Cards en vivo, gráficos volumétricos 3D de RFCs/Solicitudes, medidor de éxito (meta ≥ 95%) y dona de carga TQS.	Admin, Director, TQS	Calculado sobre dataset filtrado
En Ejecución	Monitoreo de ventanas en vivo con cronómetro restante y bloqueo estricto en caliente sobre fechas de inicio/fin/retorno.	Admin, TQS	`enforceExecutionWindowLock`
Aprobados y VOBO	Bandejas de validación técnica con checklist obligatorio (órdenes QAS, ausencia de colisiones y plan de rollback).	Admin, TQS	`PATCH /rest/v1/rfcs`
Finalizados y Cancelados	Histórico completo de cumplimiento SLA y bitácora forense con normalizador visual de coordinadores KOF vs promotores.	Todos los roles	Exportación XLSX / CSV BOM
Catálogo Sistemas & Usuarios	Gobernanza de los 54 activos KOF (criticidad 1-3, titular, backup), transferencia atómica de titularidad y gestión RBAC.	Solo Admin	`/rest/v1/sistemas` & `/auth/v1/admin`
Auditoría y Logs 360°	Consola cuádruple de eventos (Log Aplicación, Log Auditoría DB Triggers, Log Actividades y Sincronización).	Solo Admin	`/rest/v1/log_auditoria`

3. Ciclo de Vida y Gestión de Estados Globales

Al dispararse el evento `DOMContentLoaded`, la SPA ejecuta la secuencia de hidratación verificando la existencia y vigencia del token JWT en almacenamiento seguro. Si no existe sesión válida, el sistema monta de forma bloqueante la vista `loginOverlay`.

3.1 Desacoplamiento Estricto de Titularidad TQS

Directiva de Aislamiento Operacional ("Solo Mi Perfil TQS"): Para evitar que los líderes técnicos absorban registros de contingencia ajenos (por ejemplo, impidiendo que Nancy Dolores visualice los 377 RFCs de respaldo junto a sus 112 RFCs propios), la función `getUserEffectiveSystems(user)` consulta estrictamente `owner_id = user.id`. Los sistemas donde el usuario funge como

suplente se aíslan en `user.sistemas_backup` y únicamente se fusionan cuando el usuario acciona explícitamente el conmutador *Mis Sistemas & Backup*.

Adicionalmente, las guardas `isRfcOfCurrentUser(r)` y `isSolOfCurrentUser(s)` imponen la validación de propiedad directa en todas las tablas operativas y widgets del Dashboard.

4. Cliente HTTP e Interceptores (smartFetch)

Toda comunicación saliente se canaliza a través del wrapper centralizado `smartFetch(url, options)`, el cual provee:

- **Inyección Automática de Seguridad:** Adjunta la cabecera `Authorization: Bearer <jwt_token>`, `apikey` y `Content-Type: application/json` en cada transacción.
- **Normalización Semántica PostgREST:** Traduce operadores de filtrado (`eq.`, `neq.`, `in.`, `order=`, `limit=`) hacia parámetros compatibles con los endpoints Express del backend.
- **Interceptación Global de Respuestas:** Ante un código `HTTP 401` invalida el estado local y redirige al login; ante `HTTP 403` notifica restricción de privilegios RBAC sin destruir la sesión activa; ante errores `5xx` despliega alertas no bloqueantes.

5. Directivas de Producción en Nginx (nginx.conf)

El servidor web configurado dentro de `frontend/Dockerfile` aplica directivas optimizadas para entornos empresariales:

```
server {

    listen 80;

    root /usr/share/nginx/html;

    index index.html;



    # Compresion Gzip dinamica

    gzip on;

    gzip_comp_level 6;

    gzip_types text/plain text/css application/json application/javascript image/svg+xml;



    # Proxy inverso hacia el servicio atlas_backend (Docker network)

    location /auth/ { proxy_pass http://atlas_backend:54321/auth/; }

    location /rest/ { proxy_pass http://atlas_backend:54321/rest/; }

    location /api/  { proxy_pass http://atlas_backend:54321/api/;  }
```

```
    # SPA Fallback routing

    location / {

        try_files $uri $uri/ /index.html;

    }

}
```

Seguridad y Caching: Se aplican cabeceras obligatorias `X-Frame-Options: SAMEORIGIN`, `X-Content-Type-Options: nosniff`, `X-XSS-Protection: 1; mode=block` y `Referrer-Policy: strict-origin-when-cross-origin`, manteniendo una política de cache de 30 días sobre recursos estáticos e impidiendo el cacheo de respuestas dinámicas de API.